
4.4 `technical-details/unsupervised-learning.qmd`

“‘markdown

title: “Unsupervised Learning”

Unsupervised Learning

Introduction to Unsupervised Learning in the Logistics Simulation

Unsupervised learning helps spot new behavior trends in our simulated logistics setup - using live data instead of pre-labeled results. As the simulation produces constant updates like vehicle speed, location, distance covered, or task completion rates, this method reveals hidden organization in fast-moving streams of detailed information. It’s key for noticing when trips differ from norms, recognizing unusual motion sequences, while grouping vehicles based on how they perform. In our work, it supports ongoing tracking of fleets and improves later predictive systems by adding subtle but useful data cues.

Clustering Objective and Rationale

The goal here is grouping trucks by how they operate throughout a trip. Although every truck moves along the same set path between start and end points, their behaviors aren’t identical - expected speeds change according to OSRM time predictions, left-over distances shift unevenly due to differing departure times, while outside factors like rerouting or minor delays add further differences.

Clustering these operational characteristics helps us answer several important logistical questions:

- Which trucks are progressing unusually slowly or quickly?
- Are there natural groupings of vehicles based on route difficulty or distance remaining?
- Can we detect potential anomalies early by comparing a truck’s behavior to its cluster peers?
- How do operational factors naturally segment across different routes?

Unsupervised segmentation provides actionable insights even in a simulated setting, aligning closely with industry fleet-monitoring practices.

Feature Selection for Clustering

We cluster each truck using two continuous operational features, chosen for interpretability and real-time relevance:

1. Planned Speed (km/h) Derived from OSRM's estimated travel duration, this value predicts how fast a truck should move over the entire route. Differences arise from route geometry, road quality, and length.
2. Remaining Distance (km) Calculated as:

remaining = total_distance_km − cumulative_distance_traveled This feature indicates where the truck is in its journey and distinguishes early-route trucks from those nearing completion.

Together, these features capture:

- Expected vs. actual performance potential
- Progress dynamics
- Proximity to destination
- Route-specific difficulty

They also produce well-separated clusters that are meaningful in real operations.

Clustering Methodology: K-Means

We employ K-Means clustering, a widely used algorithm for partitioning numerical data into k distinct groups based on similarity.

Training procedure: A dataset of simulated truck journeys was collected across multiple routes.

For each logged timestamp, planned_speed_kmh and dist_remaining_km were extracted.

The data was normalized (if necessary) to stabilize cluster boundaries.

K-Means was fit with k = 3, producing the following conceptual clusters:

Cluster	Interpretation	Behavioral Pattern
0 – Long-Distance / Early Journey	High remaining distance, moderate speed	Truck has just begun its route
1 – Mid-Route / Steady State	Moderate distance remaining	Most trucks spend the longest time here
2 – Arrival Phase / Low Distance	Nearly complete	Truck is close to the depot and progress stabilizes

Real-Time Application in the Simulation

Once deployed, the trained model is loaded by the backend and applied to each truck every simulation tick:

cluster = kmeans_model.predict([[plan_speed_kmh, dist_remaining]])[0] The assigned cluster is then included in the WebSocket payload and displayed on the frontend map inside each truck's information popup.

This enables: Live monitoring of fleet behavior

- Pattern detection (e.g., which clusters appear most often)
- Quick visual segmentation of trucks along the route
- Future anomaly detection (trucks behaving differently from cluster peers)

Even though this is a simulated environment, the clustering system behaves exactly like one used in real-world fleet analytics.

Insights Gained from Clustering

During EDA and deployment, clustering revealed several useful insights:

- Trucks with longer or more complex OSRM routes tend to fall into higher-distance clusters early on.
- Variability in planned speed often creates distinct cluster boundaries even when trucks are the same distance from the destination.
- The cluster IDs provide a simple but effective indicator of operational phase: beginning, middle, or near completion.
- These cluster features became important inputs to subsequent supervised models (e.g., predicting delay risk).

Limitations and Considerations

While K-Means is simple and effective, several limitations need to be acknowledged:

- Clusters are spherical and may not capture nonlinear route behaviors.
- Synthetic data reduces variability, which can make clusters appear cleaner than in reality.
- The model must be retrained if new features are introduced or if simulation behavior changes drastically.

- Scaling issues may arise if the feature magnitudes differ significantly.

In future extensions, more advanced unsupervised methods—such as DBSCAN for anomaly detection or Gaussian Mixture Models for probabilistic segmentation—could enrich system insights.

Conclusion

Using unsupervised methods helped uncover hidden patterns in the simulated delivery data. Because trucks were grouped by target speed and left-over distance, insights became clearer during live operation. Such grouping links actual simulation output with later prediction models that rely on labeled examples.